

浙江大学

本科实验报告

课程名称：计算机组成与设计

姓名：张序鸿

学院：竺可桢学院

专业：计算机科学与技术

学号：3230103265

指导教师：杨莹春

2024 年 11 月 15 日

浙江大学实验报告

课程名称：____计算机组成与设计____ 实验类型：____综合____

实验项目名称：____CPU 设计之控制器____

学生姓名：____张序鸿____ 专业：____混合班____ 学号：____3230103265____

指导老师：____杨莹春、潘之杰____ 助教：____张立冬、马致远____

实验地点：____东 4-509____ 实验日期：____2024 年 11 月 20 日____

一、实验目的和要求

1. 复习寄存器传输控制技术
2. 掌握 CPU 的核心：指令执行过程与控制流关系，并设计 Control Unit
3. 设计控制器
4. 学习测试程序的设计

二、实验内容和原理

内容：

任务一：用硬件描述语言设计实现控制器

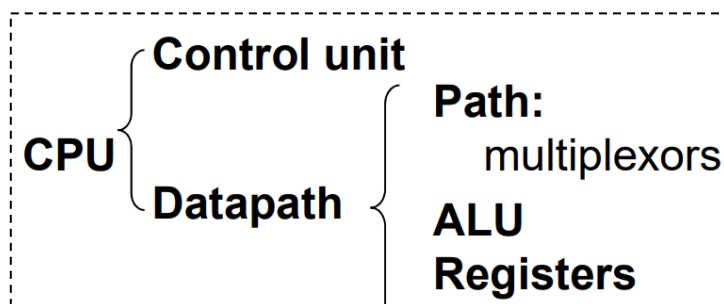
——根据 Exp04-1 数据通路及指令编码完成控制信号真值表

任务二：设计控制器测试方案并完成测试

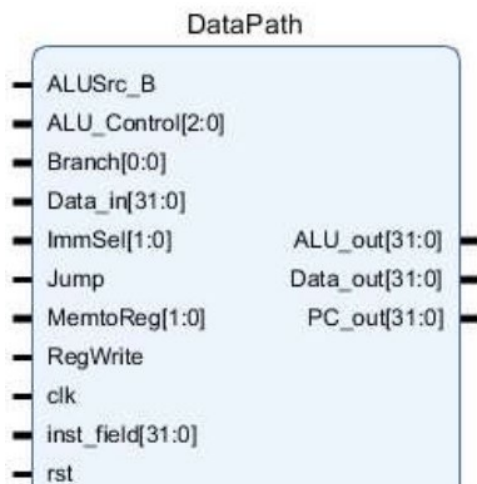
——OP 译码测试：R-格式、访存指令、分支指令，转移指令

原理：

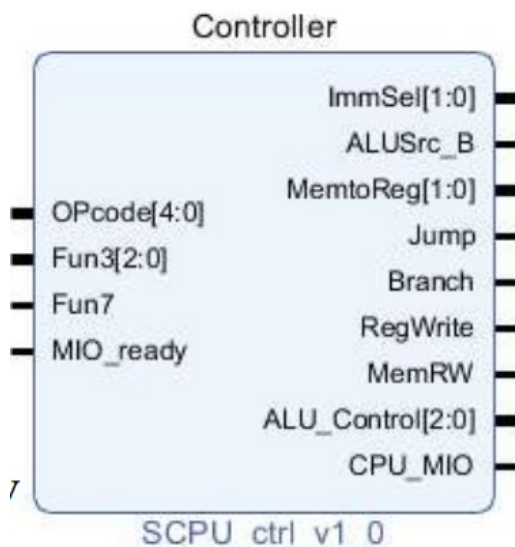
2.1 计算机系统中 CPU 组成



- 数据通路 Data_path 作为 CPU 主要部件之一，具有通用计算功能的算术逻辑部件（主要为 ALU）、具有通用目的寄存器堆、具有通用计数所需的尽可能的路径。



- 控制器 SCPU_ctrl 是寄存器传输控制技术中的运算和通路控制器，它起到指令译码（inst_in 的分块）、产生操作控制信号（ALU 运算控制）、产生指令所需的路径选择的作用。



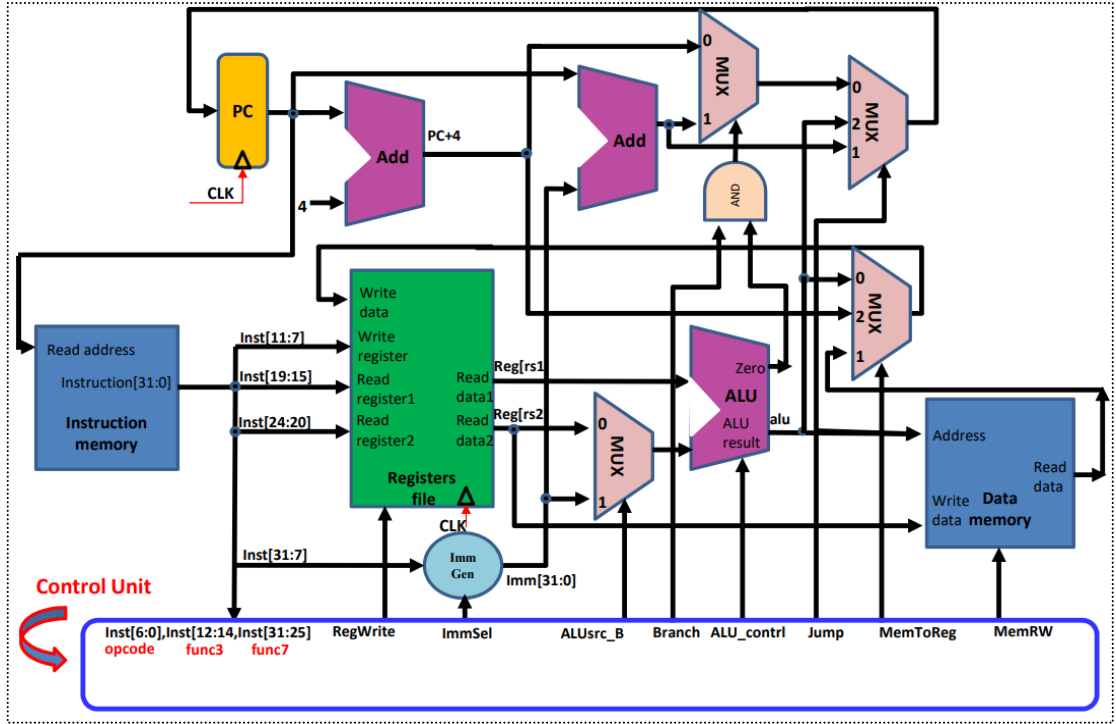
2.2 RISC-V 控制器 SCPU_ctrl 原理介绍

控制单元作为处理器的一部分，用以告诉数据通路需要做什么，包含了取指单元（PC 及地址计算单元）、译码单元、控制单元（时序信号形成及微操作控制信号形成电路）、中断等。

通路和操作控制信号如下：

信号	源数目	功能定义	赋值0时动作	赋值1时动作	赋值2时动作
ALUSrc_B	2	ALU端口B输入选择	选择源操作数寄存器2数据	选择32位立即数（符号扩展后）	-
MemToReg	3	寄存器写入数据选择	选择ALU输出	选择存储器数据	选择PC+4
Branch	2	Beq指令目标地址选择	选择PC+4地址	选择转移目的地址PC+imm（zero=1）	-
Jump	3	J指令目标地址选择	由Branch决定输出	选择跳转目标地址PC+imm（JAL）	-
RegWrite	-	寄存器写控制	禁止寄存器写	使能寄存器写	-
MemRW	-	存储器读写控制	存储器读使能，存储器写禁止	存储器写使能，存储器读禁止	-
ALU_Control	000-111	3位ALU操作控制	参考表ALU_Control		
ImmSel	000-111	3位立即数组合控制	参考表ImmSel		

【具体操作指令的具体信号译码在此不给出，详见后续实验部分】



上图给出了各控制信号在数据通路中的控制线路和作用。

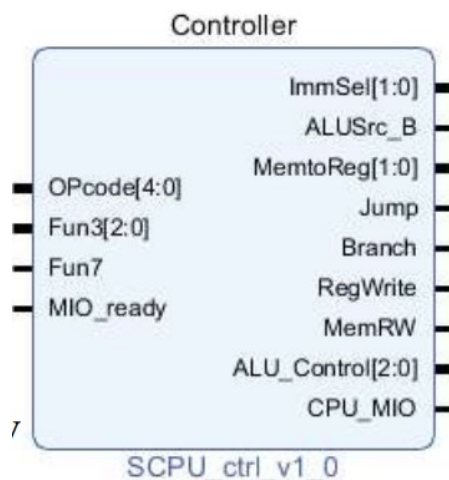
三、实验过程和数据记录

3.1 设计实现 SCPU_Ctrl:

(1) Vivado 新建工程

- 利用 Vivado 新建工程，命名 OExp04-SCPU_ctrl；
- 最后正确选择 FPGA 芯片——xc7a100tcs9324-1；

本实验主要实现两个阶段的译码：根据指令 inst_in 对主要控制信号进行赋值分配；根据 ALU_op、Funct3/7 对 ALU 中的 ALU_Control 进行赋值。



(2) 描述设计 SCPU_ctrl:

- 逻辑接口:

```
module SCPU_ctrl(
    input[4:0]OPcode, //Opcode-----inst[6:2]
    input[2:0]Fun3, //Function-----inst[14:12]
    input Fun7, //Function-----inst[30]
    input MIO_ready, //CPU Wait
    output reg [1:0]ImmSel, //立即数选择控制
    output reg ALUSrc_B, //源操作数 2 选择
    output reg [1:0]MemtoReg, //写回数据选择控制
    output reg Jump, //jal
    output reg Branch, //beq
    output reg RegWrite, //寄存器写使能
    output reg MemRW, //存储器读写使能
    output reg [2:0]ALU_Control, //alu 控制
    output reg CPU_MIO
);
reg [1:0] ALUOp;
wire [3:0] Fun;
reg [10:0]CPU_ctrl_signals;
```

- 主控制器设计:

Inst[31:0]	Banch	Jump	ImmSel	ALUSrc_B	ALU_Control	MemRW	RegWrite	MemtoReg
add	0	0	*	Reg	Add	Read	1	ALU
sub	0	0	*	Reg	Sub	Read	1	ALU
(R-R Op)	0	0	*	Reg	(Op)	Read	1	ALU
addi	0	0	I	Imm	Add	Read	1	ALU
lw	0	0	I	Imm	Add	Read	1	Mem
sw	0	0	S	Imm	Add	Write	0	*
beq	0	0	B	Reg	Sub	Read	0	*
beq	1	0	B	Reg	sub	Read	0	*
jal	0	1	J	Imm	*	Read	1	PC+4
lui	0	0	U	Imm	Add	Read	1	ALU

【各类型指令主控制信号真值表】

```

`define CPU_ctrl_signals
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,Jump,ALUop,ImmSel}
  always @(*) begin
    case(OPcode)
      5'b01100: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,Jump,ALUop,ImmSel} =
11'b0_00_1_0_0_0_10_00; end //ALU    - R
      5'b00000: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,Jump,ALUop,ImmSel} =
11'b1_01_1_0_0_0_00_00; end //load    - I
      5'b01000: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,Jump,ALUop,ImmSel} =
11'b1_00_0_1_0_0_00_01; end //store    - S
      5'b11000: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,Jump,ALUop,ImmSel} =
11'b0_00_0_0_1_0_01_10; end //beq     - B
      5'b11011: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,Jump,ALUop,ImmSel} =
11'b0_10_1_0_0_1_00_11; end //jump    - J
      5'b00100: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,Jump,ALUop,ImmSel} =
11'b1_00_1_0_0_0_11_00; end //ALU    - I
      default:
begin {ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,Jump,ALUop,ImmSel} =
11'b000000000000; end
    endcase
  end

```

- ALU 译码控制设计:

Instruction opcode	op	Instruction operation	Funct 3	Funct7	ALUop	Desired ALU action	ALUControl
R-type	0110011	add	000	0000000	10	add	010
		sub	000	0100000		sub	110
		sll	001	0000000		sll	-
		slt	010	0000000		slt	111
		slltu	011	0000000		slltu	-
		xor	100	0000000		xor	011
		srl	101	0000000		srl	101
		sra	101	0100000		sra	-
		or	110	0000000		or	001
		and	111	0000000		and	000

ALU 相关操作对应的控制信号如上图:

```
assign Fun = {Fun3, Fun7};

always @(*) begin
    case(ALUop)
        2'b00: ALU_Control = 3'b010 ; //add
        2'b01: ALU_Control = 4'b110 ; //sub
        2'b10:
            case(Fun)
                4'b0000: ALU_Control = 3'b010 ; //add
                4'b0001: ALU_Control = 3'b110 ; //sub
                4'b1110: ALU_Control = 3'b000 ; //and
                4'b1100: ALU_Control = 3'b001 ; //or
                4'b0100: ALU_Control = 3'b111 ; //slt
                4'b1010: ALU_Control = 3'b101 ; //srl
                4'b1000: ALU_Control = 3'b011 ; //xor
                default: ALU_Control = 3'bx;
            endcase
        2'b11:
            case(Fun3)
                3'b000: ALU_Control = 3'b010 ; //addi
                3'b010: ALU_Control = 3'b111 ; //slli
                3'b100: ALU_Control = 3'b011 ; //xori
                3'b110: ALU_Control = 3'b001 ; //ori
                3'b111: ALU_Control = 3'b000 ; //andi
                3'b101: ALU_Control = 3'b101 ; //srli
                default: ALU_Control = 3'bx ;
            endcase
    endcase
endcase
end
```

(3) SCPU_ctrl 仿真测试:

SCPU_ctrl 行为仿真, 对 I、R、S、B、J 型指令的控制信号进行了验证, 主要检查对应 Opcode 下各个控制信号是否符合设计要求。

• 仿真代码如下:

```
`timescale 1ns / 1ps

module SCPU_tb();
    reg[4:0] OPcode;
    reg MIO_ready;
    reg [2:0]Fun3;
    reg Fun7;
    reg Fun;
    wire [1:0]ImmSel;
    wire ALUSrc_B;
    wire Jump;
    wire [1:0] MemtoReg;
    wire Branch;
    wire RegWrite;
    wire MemRW;
    wire [2:0]ALU_Control;
    wire CPU_MIO;

    SCPU_ctrl U1(
        .OPcode(OPcode),
        .Fun3(Fun3),
        .Fun7(Fun7),
        .MIO_ready(MIO_ready),
        .ImmSel(ImmSel),
        .ALUSrc_B(ALUSrc_B),
        .MemtoReg(MemtoReg),
        .Jump(Jump),
        .Branch(Branch),
        .RegWrite(RegWrite),
        .MemRW(MemRW),
        .ALU_Control(ALU_Control),
        .CPU_MIO(CPU_MIO)
    );
endmodule
```

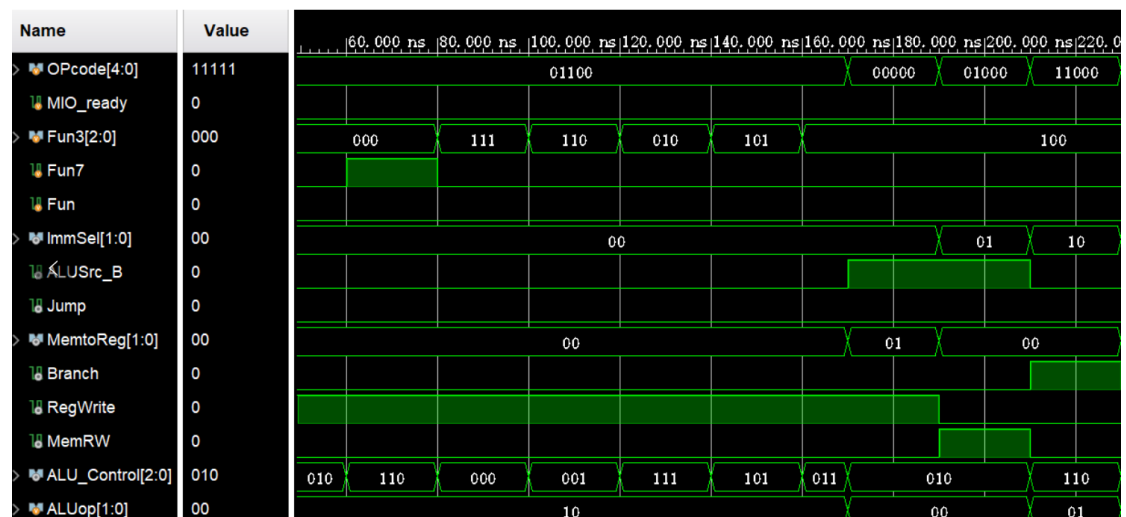


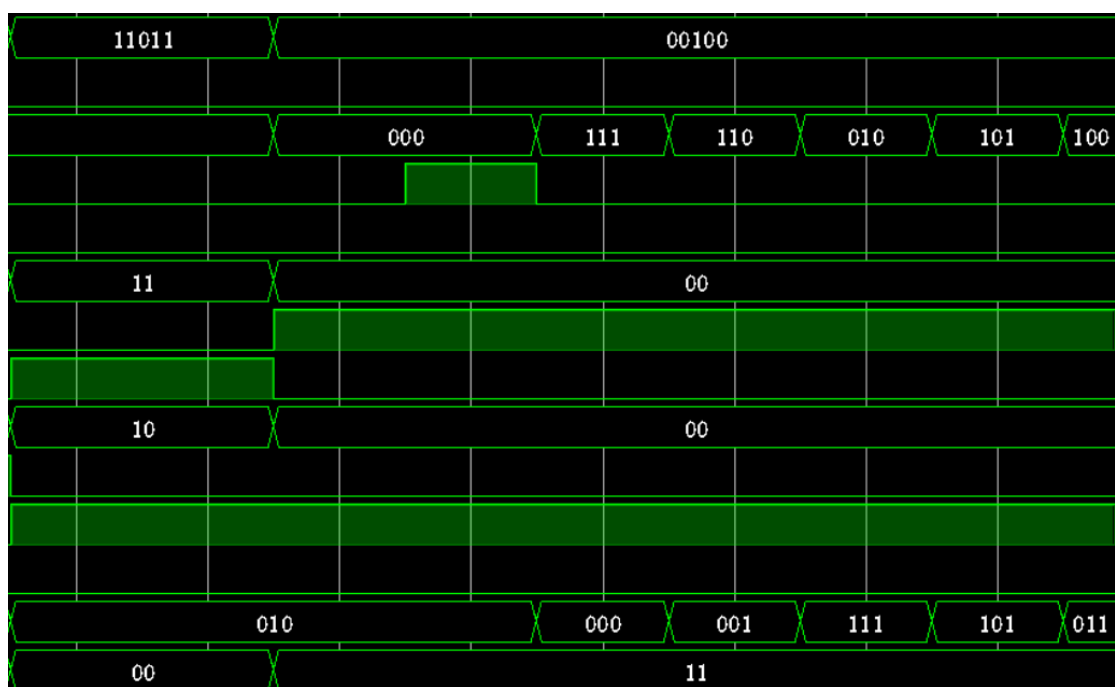
```

initial begin
    OPCODE = 0;
    Fun = 0;
    Fun3 = 0;
    Fun7 = 0;
    MIO_ready = 0; #40; // Wait 40 ns for global reset to finish

    OPCODE = 5'b01100;          //ALU 指令,检查 ALUOp=2'b10; RegWrite=1
    Fun3 = 3'b000; Fun7 = 1'b0;#20; //add,检查 ALU_Control=3'b010
    Fun3 = 3'b000; Fun7 = 1'b1;#20; //sub,检查 ALU_Control=3'b110
    Fun3 = 3'b111; Fun7 = 1'b0;#20; //and,检查 ALU_Control=3'b000
    Fun3 = 3'b110; Fun7 = 1'b0;#20; //or,检查 ALU_Control=3'b001
    Fun3 = 3'b010; Fun7 = 1'b0;#20; //slt,检查 ALU_Control=3'b111
    Fun3 = 3'b101; Fun7 = 1'b0;#20; //sr1,检查 ALU_Control=3'b101
    Fun3 = 3'b100; Fun7 = 1'b0;#10; //xor,检查 ALU_Control=3'b011
    OPCODE = 5'b00000; #20//load 指令,检查 ALUOp=2'b00// ALUSrc_B=1,
MemtoReg=1, RegWrite=1
    OPCODE = 5'b01000; #20; //store 指令,检查 ALUOp=2'b00, MemRW=1,
ALUSrc_B=1
    OPCODE = 5'b11000; #20//beq 指令,检查 ALUOp=2'b01, Branch=1 #20;
    OPCODE = 5'b11011; #40//jump 指令,检查 Jump=1
    OPCODE = 5'b00100; //I 指令,检查 ALUOp=2'b11; RegWrite=1 #20;
    Fun3 = 3'b000; Fun7 = 1'b0;#20; //addi,检查 ALU_Control=3'b010
    Fun3 = 3'b000; Fun7 = 1'b1;#20; //subi,检查 ALU_Control=3'b110
    Fun3 = 3'b111; Fun7 = 1'b0;#20; //andi,检查 ALU_Control=3'b000
    Fun3 = 3'b110; Fun7 = 1'b0;#20; //ori,检查 ALU_Control=3'b001
    Fun3 = 3'b010; Fun7 = 1'b0;#20; //slti,检查 ALU_Control=3'b111
    Fun3 = 3'b101; Fun7 = 1'b0;#20; //srli,检查 ALU_Control=3'b101
    Fun3 = 3'b100; Fun7 = 1'b0;#20; //xori,检查 ALU_Control=3'b011
    OPCODE = 5'h1f; //间隔
    Fun3 = 3'b000; Fun7 = 1'b0; //间隔
end

```

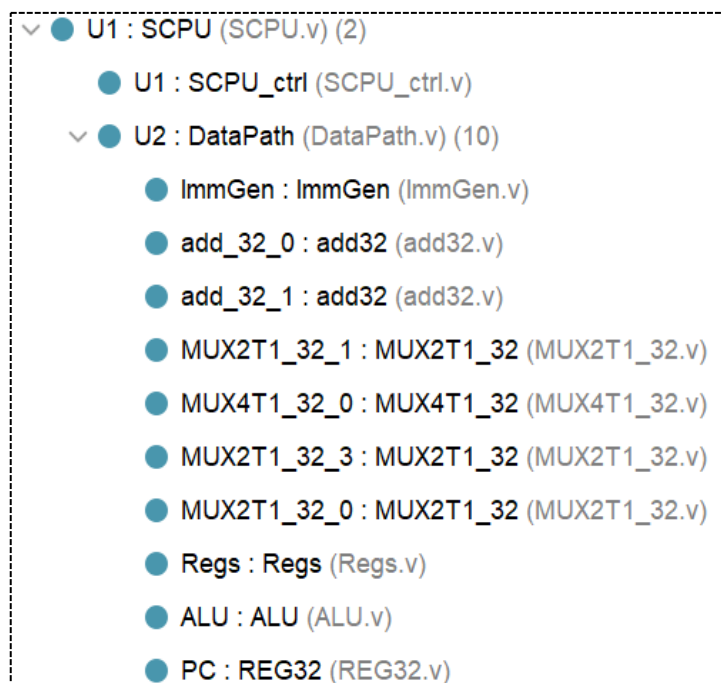




控制信号的具体赋值在仿真代码中注释，可以看到仿真结果与预期结果一致，SCPU_ctrl 的设计正确。

3.2 集成 CPU 并搭建仿真平台

通过 Lab4-01 和 Lab4-02 的实验，我们通过逻辑代码实现了 Datapath 和 SCPU_ctrl 模块，因此我们可以集成得到一个 CPU。



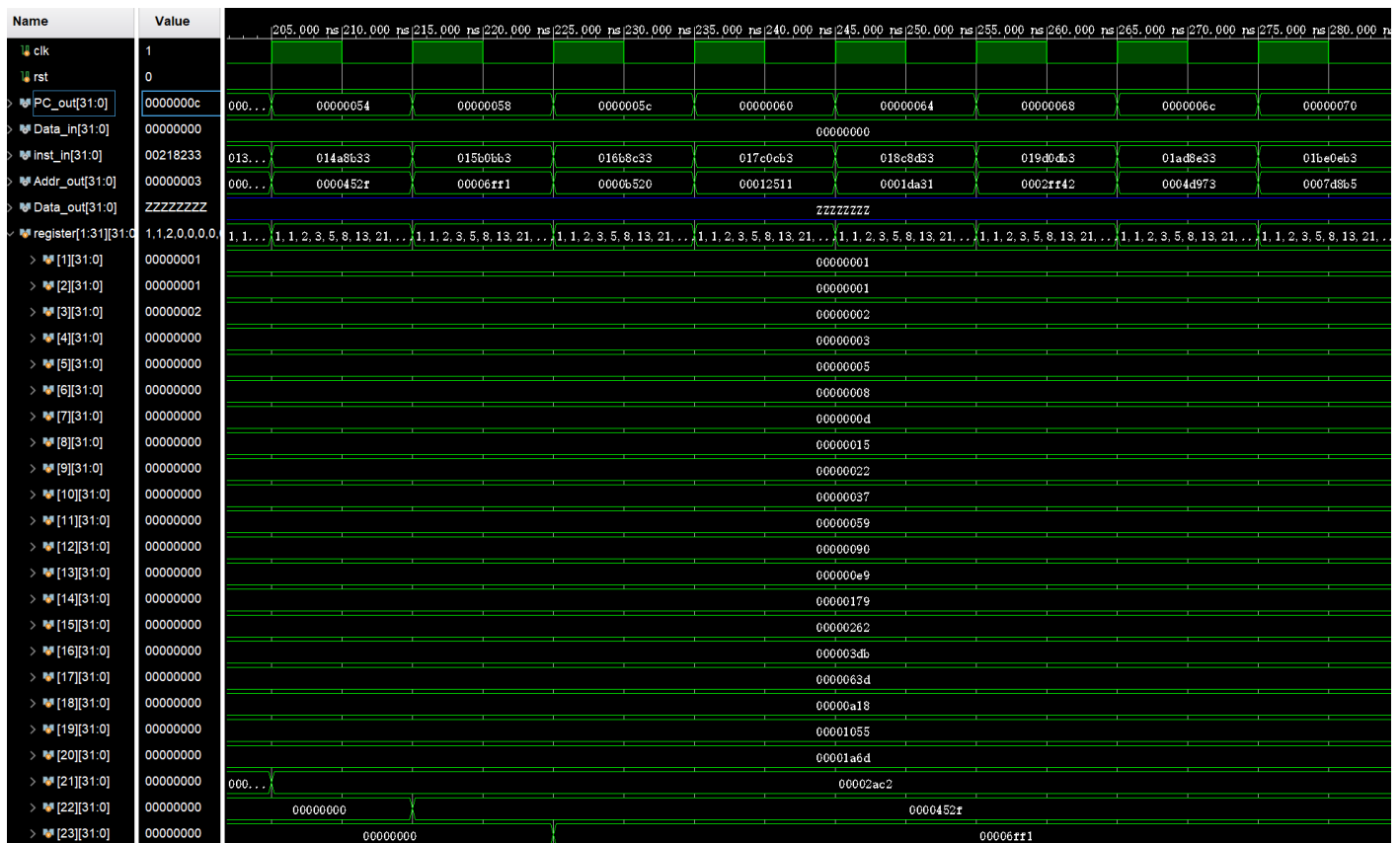
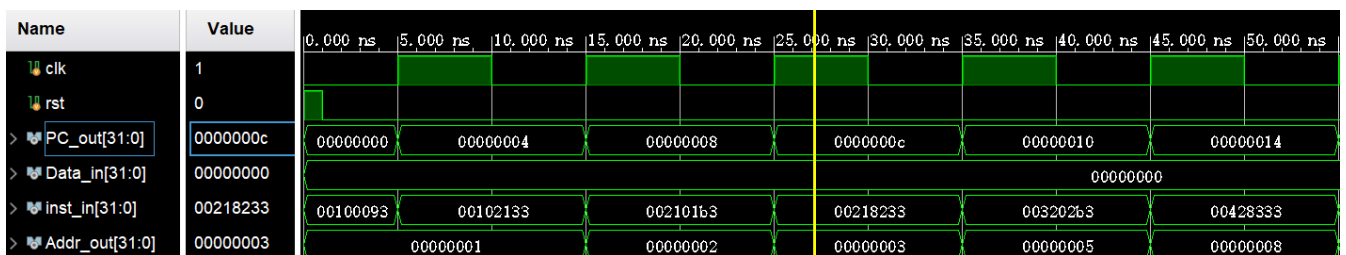
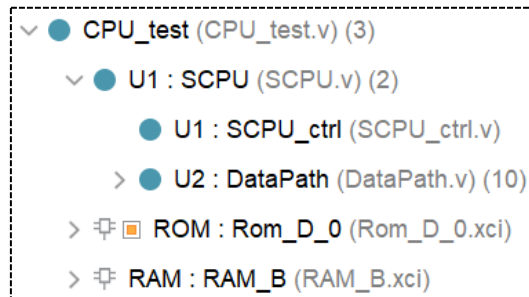
【具体代码如下】

```

module SCPU(
    input clk,
    input rst,
    input [31:0]Data_in,
    input [31:0]inst_in,
    input MIO_ready,
    output MemRW,
    output CPU_MIO,
    output [31:0]Data_out,
    output [31:0]PC_out,
    output [31:0]Addr_out
);
    wire [2:0]ALU_Control;
    wire [1:0]ImmSel , MemtoReg;
    wire Jump, Branch, RegWrite, ALUSrc_B;
    SCPU_ctrl U1(
        .OPcode(inst_in[6:2]),
        .Fun3(inst_in[14:12]),
        .Fun7(inst_in[30]),
        .MIO_ready(MIO_ready),
        .ImmSel(ImmSel),
        .ALUSrc_B(ALUSrc_B),
        .MemtoReg(MemtoReg),
        .Jump(Jump),
        .Branch(Branch),
        .RegWrite(RegWrite),
        .ALU_Control(ALU_Control),
        .CPU_MIO(CPU_MIO) );
    DataPath U2(
        .ALUSrc_B(ALUSrc_B),
        .ALU_operation(ALU_Control),
        .Branch(Branch),
        .Data_in(Data_in),
        .ImmSel(ImmSel),
        .Jump(Jump),
        .MemtoReg(MemtoReg),
        .RegWrite(RegWrite),
        .clk(clk),
        .inst_field(inst_in),
        .rst(rst),
        .ALU_out(Addr_out),
        .Data_out(Data_out),
        .PC_out(PC_out));
endmodule

```

利用 CPU（本实验设计）、RAM、ROM 可以搭建如下图的仿真平台，初始化 RAM 的数据（D_mem.coe），初始化 ROM 的指令（I_mem.coe），可以完成 CPU 各种类型指令的执行，查看 CPU 数据的输入和输出，以及底层的包括寄存器在内的个模块的状态。



四、实验结果分析

CPU 集成时采用的仿真指令仍旧采用了 Lab2 中的 I_mem.coe，主要是为了验证自主集成的 CPU 是否能够进行正常的指令读取与简单操作（指令完备性在 ReadmeLab4.txt 中提到会在 Lab4-03 的 DEMO 中体现，在此不重复验证）。

3.2 中的图一可以看出 PC 可以正常+4 进行下一指令的读取，同时 addr_out = alu_res 可以看出 ALU 的计算结果。

图二则是对寄存器组的观察，可以看到寄存器组能够正常更新。

从仿真结果来看，Lab4-00/01/02 共同实现的简单 CPU 的功能没有问题！

五、讨论与心得

思考题 1：单周期控制器时序体现在哪里？

“always@后面内容是敏感变量，always@()里面的敏感变量为*，意思是说敏感变量由综合器根据 always 里面的输入变量自动添加。”*

```
always @(*) begin
    case(OPcode)
    endcase
always @(*) begin
    case(ALUop)
    endcase
end
```

单周期控制器中采用二级译码，第一级的敏感变量是 Opcode（唯一输入），而且 Opcode 是每时序周期根据指令修改，因此完成一级译码。

二级译码中的 ALUop 则是一级译码的结果，也是在每一周期中进行判断，从而输出 ALU_Control。

思考题 2：设计 bne 指令需要增加控制信号吗？

需要添加一个类似 Branch 的控制信号 BranchN。

Beq 判定跳转的条件是 zero & Branch = 1（利用一个 and），如果不添加控制信号，Branch = 1、zero = 0 无法实现 bne 跳转。

因此添加 BranchN，在判定 PC 跳转的 MUX 处，添加一个 or，使得 BranchN = 1、zero = 0 时也能够利用 BranchN & (! zero) 来判定 bne 的跳转。

思考题 3:

(1) 扩展下列指令, 控制器将作如何修改: **R-Type: sra, sll, sltu; I-Type: srar, sllr, sltur ; B-Type: bne; U-Type: lui; ?**

增加 R-Type、I-Type 时, 由于指令超过 8 个, ALU_Control 需要修改为 4 位, 因此第二级 ALU 译码时需要修改。

增加了 B-Type、U-Type, 因此需要在译码时增加 BranchN 信号、同时修改主控制器 (一级译码), 利用新的 Opcode 来编码 U-Type。

(2) 此时利用二级译码有优势吗?

二级译码让增加新的指令、以及修改译码模式都较轻松。

Lab4 到本实验为止, 完成了简单 CPU 的设计与集成, 设计的时候比较麻烦的是组件模块与线路略微复杂, 导致会出现一些小错误的出现, 需要注意。

但是。。。指令拓展和中断处理还在后头, 需要对 Datapath 和 SCPU_ctrl 的设计过程比较熟悉, 否则在添加新的信号时容易摸不着头脑。